CASE STUDY · SOLO BUILD

The stuff you save to get better at your job, and never look at again.

How I designed and built an AI-powered personal knowledge system. The first version focuses on Product Management learners as its MVP audience; the architecture is built to support any learning domain.

Everyone saves things they intend to learn from, and almost nobody goes back to them. PM Content Vault — the MVP implementation of a broader knowledge system — takes the posts, PDFs, screenshots and chat exports scattered across five apps, works out what each one is, and lets you ask questions of the whole pile, with every answer citing the saves it came from.

Role Product & build, end to end **Status** Live

Try it without signing up — `demo@pmvault.app` / `PMVaultDemo2026!`
A demo account with one saved item of every supported type.

The problem

Anyone learning something on their own accumulates a trail of saved material — a framework posted on LinkedIn, a PDF shared in a group chat, a screenshot of a diagram, a long article bookmarked at midnight. It is saved with real intent, and almost none of it is read twice.

The obvious diagnosis is that people are disorganised, and the obvious fix is a better folder structure. I don't think that's right. The saving isn't the broken part — people save diligently. Three things break afterwards:

It's scattered
LinkedIn saves, WhatsApp media, browser bookmarks, a Notes app. No single place to look, so looking anywhere feels pointless.

You can't judge it
A saved link is a title and a thumbnail. Whether it's worth ten minutes takes ten minutes to find out.

You can't find it
You remember *"that post about handling stakeholder conflict"* — not a keyword that appears in it.

THE INSIGHT THIS PRODUCT IS BUILT ON

Storage was never the bottleneck. **Retrieval and judgment** were. A folder app solves the problem I don't have. That single conclusion is why summaries are generated and stored at ingest rather than on demand, and why semantic search came before organisation features.

Who it's for

Ultimately, anyone who saves material to learn from later — people studying for exams, switching careers, ramping into a new role, or teaching themselves a craft. The behaviour is the same everywhere: save with intent, never return.

The first user segment is one large, well-defined slice of that: **Product Management learners.** Not because the problem is unique to them, but because they have a shared vocabulary, a shared deadline and predictable content types — which makes the classifier tractable, the taxonomy meaningful, and the users reachable for research. It is the initial learning domain, not the ceiling.

WHY THE MVP STARTS NARROW WHEN THE PROBLEM IS BROAD

A fixed taxonomy is what makes saved content instantly scannable, and a taxonomy only works if you know the domain. Building for everyone on day one means either no categories or generic ones, and both put the burden of organising back on the user — which is the thing that already failed. **Generalising is a configuration problem, not a rebuild**: the ingestion, embedding, retrieval and citation machinery is domain-agnostic already. Only the category list is not.

Why build it, when so much of this exists

Plenty of tools hold saved content. None of them were built for the specific failure I had, and it's worth being precise about why rather than waving at "existing tools don't do it".

WHAT I COULD HAVE USED	WHAT IT'S BUILT FOR — AND WHERE THAT LEAVES ME
A notes app	Excellent storage, and organising is the user's job. That's precisely the step that gets skipped at the moment of saving, which is when you're mid-scroll and least willing to file anything. Any tool that asks me to organise recreates the problem.
A read-later app	Built around articles and a reading queue. Most of what I save isn't an article — it's a screenshot, a PDF, a chat message, a post. And it still answers "what have I not read", not "where's that thing about stakeholder conflict".
A highlights tool	Strong at capturing passages out of things you read. It still assumes you know which source to go back to; the search is over sources, and my problem starts before I know which source it is.

WHAT I COULD HAVE USED	WHAT IT'S BUILT FOR — AND WHERE THAT LEAVES ME
A general AI chatbot	Can absolutely answer questions about a document you hand it. The gap is persistence — every session starts empty, so the library never accumulates, nothing gets deduplicated, and nothing carries state like "already read" or "starred". A conversation isn't a corpus.
Platform search	Saved posts stay on the platform that hosted them, so there are five searches instead of one — each keyword-only, none aware of the others.

THE GAP THIS FILLS

Every option above is strong at one of *accept anything*, *understand it without being asked*, or *answer across all of it with citations*. The product exists because the value is in doing all three to the same pile — the understanding has to happen at save time, or the retrieval has nothing to work with later.

The journey, and where it still hurts

Four moments matter. Each one names what the product does and, honestly, what friction is left.

1

Encounter — something worth keeping, mid-scroll

On a phone, inside LinkedIn or a group chat. The window for saving is a few seconds wide; anything more elaborate and the thing is lost.

Still friction: copy → switch app → paste

This is the one step the MVP doesn't fix, and the reason share-sheet ingestion is first on the roadmap.

2

Save — hand it over and stop thinking

Paste, upload or drop in a chat export. Extraction, classification, summarisation, embedding and a duplicate check all run before anything is written, and the result is shown for approval with every field editable.

No filing, no folders, no naming

3

Return — days later, half-remembering

Either browse the Library, where each card carries a gist, a type and a read time so relevance is judgeable without opening anything — or ask a question in plain language and let retrieval find it.

Semantic search, not keyword recall

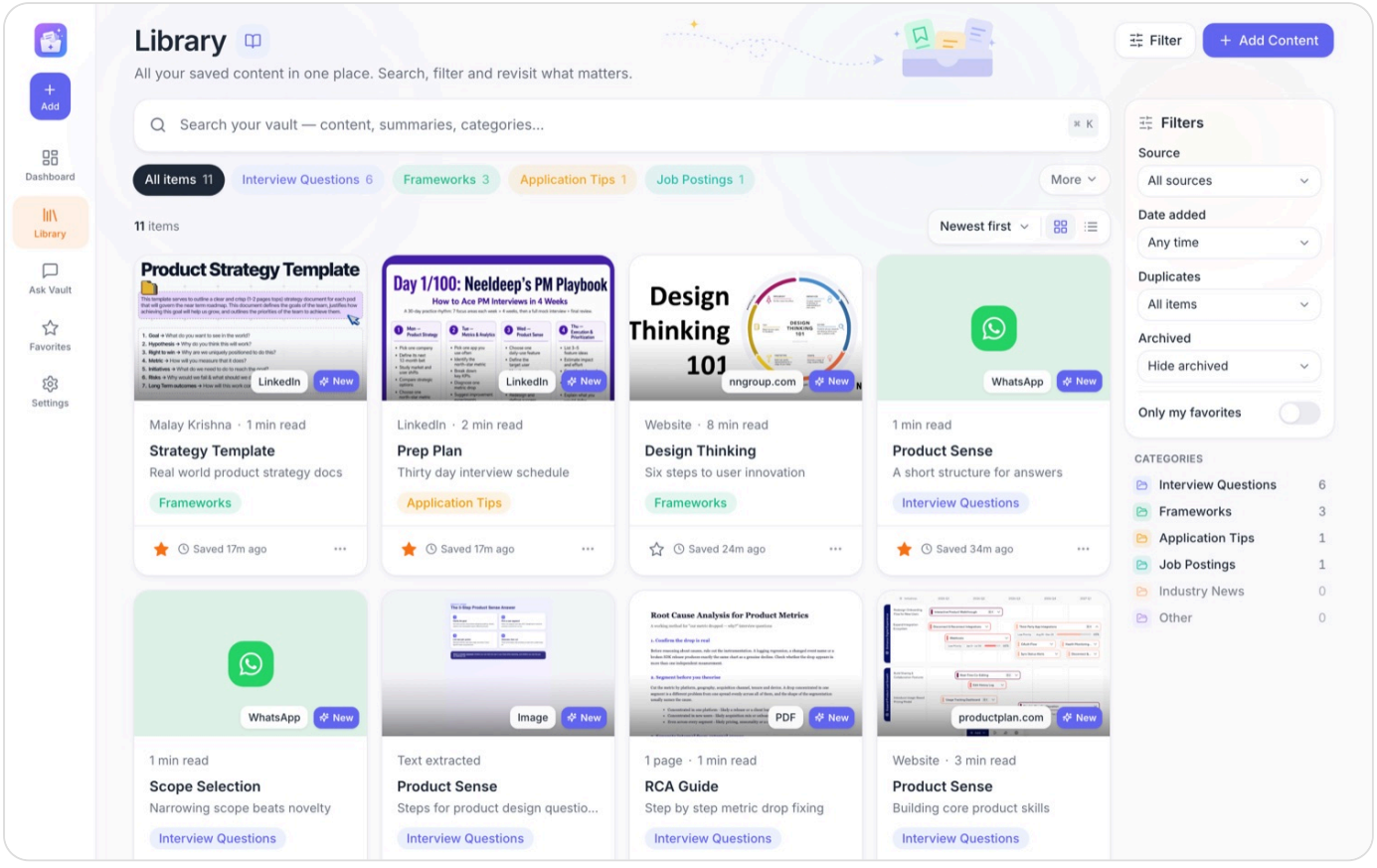
4

Verify — decide whether to trust the answer

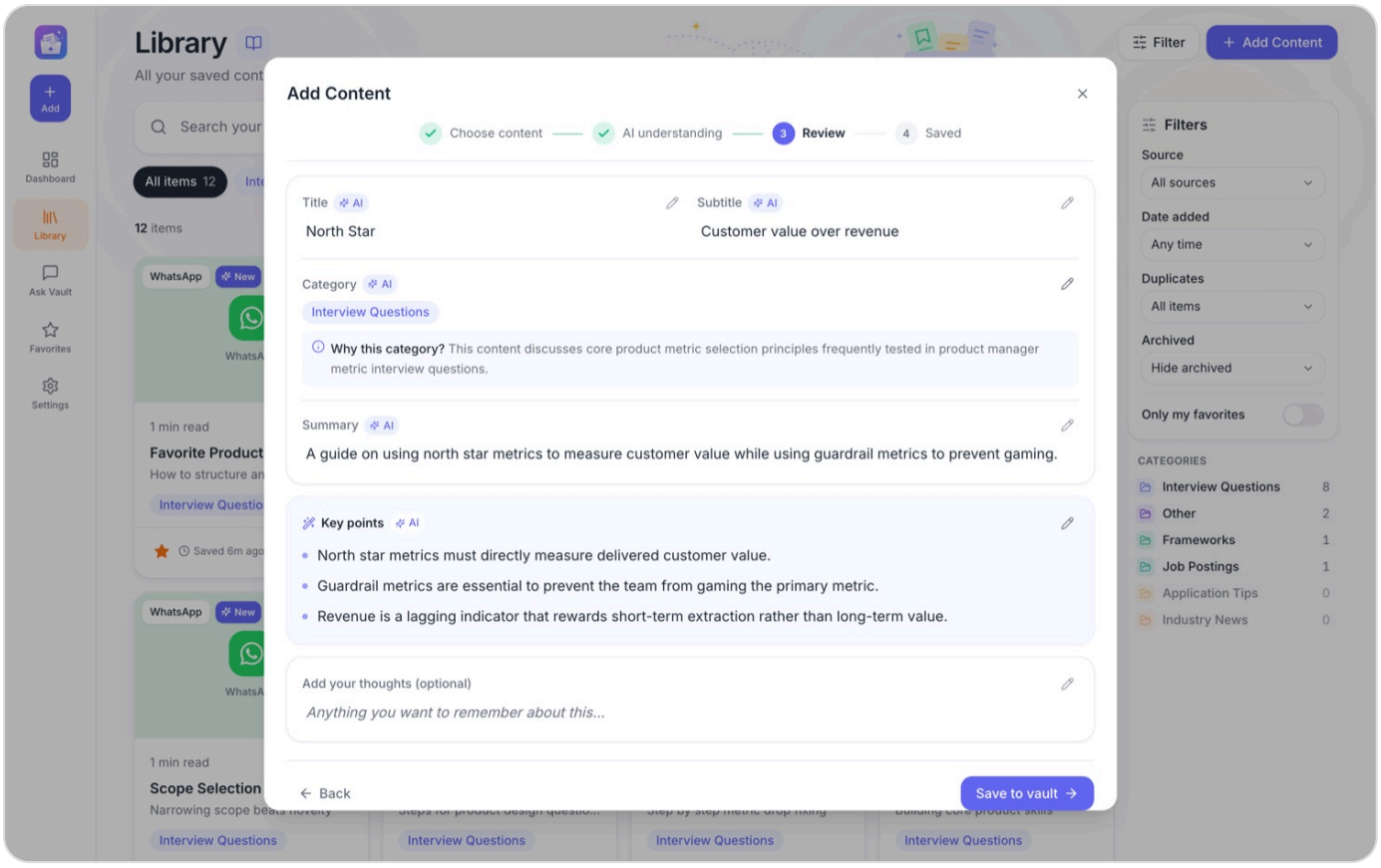
Every answer carries numbered citations. Expanding one shows the exact passage it drew on; opening it lands on the original save, where it can be read, highlighted and annotated.

Checkable, not just plausible

What it does



Every card knows what it is. A PDF shows its rendered first page, a screenshot shows the image, a LinkedIn post shows the author and its preview, a chat import shows the WhatsApp mark. Nothing here is a generic file icon — you should recognise a save without reading it. Metrics like page count and read time come from stored values, and a card omits any it doesn't have rather than printing a zero.



The AI proposes, you approve. Adding something runs extraction, classification, summarisation, embedding and a duplicate check — then stops and shows you the result. Every field is editable, and an `AI` chip marks what the model wrote and disappears the moment you change it.

Answers you can check. Ask My Vault retrieves from your own saves and answers in structured sections, with inline markers pointing at numbered sources. Expanding a source shows the exact passage the answer drew on — not the item's summary, which is a different sentence written for a different purpose.

Built for where saving actually happens. The sidebar collapses to a bottom bar; cards reflow to a single column.

Information architecture

Five destinations, and one decision underneath them that everything else follows from.

```
Dashboard      — counts, recent saves, continue a conversation
Library        — everything, filtered by category / source / date / duplicates
  └ Item       — the content, its AI summary, highlights, related items
Ask My Vault   — conversations, each answer cited back into the Library
Favorites      — the starred subset, with study actions over it
Settings       — account, defaults, data
```

The item is the atom

A WhatsApp export doesn't become one saved item — it becomes one item *per message*. A 40-message chat holding three useful ideas would otherwise be a single unsearchable blob, and retrieval would return the whole conversation when you wanted one paragraph of it. Splitting at ingest is what lets a citation point at something small enough to check.

Two kinds of label, on purpose

	CATEGORIES	TAGS
Who sets them	The AI, from a fixed list; the user can override	The user, freely
How many	One to three per item, never zero	Any
Why	A closed list is what makes the filter rail trustworthy — an open one produces "Product Sense", "Product-Sense" and "PS" within a week	Personal structure the taxonomy can't anticipate; favourites are implemented as one

NEVER ZERO

Every item lands in at least one category, falling back to `Other` if the model won't commit. An uncategorised bucket sounds harmless and is where things go to be forgotten — which is the entire problem this product exists to fix.

Decisions that shaped it

The interesting part of this project isn't the feature list — it's what got argued down.

DECISION 01

RAG, not fine-tuning — and not keyword search

Three options for "find the right thing fast". Keyword search fails on the exact query people actually have, because you remember a post's *idea*, not its vocabulary. Fine-tuning a model on a personal library that changes every day means retraining to make one new save findable — the cost is wrong for the use case.

Retrieval-augmented generation fits the shape of the problem: a new save is queryable the moment it's embedded, and answers are grounded in the user's own material with citations attached. **The point is trustworthy recall of your own content, not general knowledge.** That framing became the tiebreaker for later decisions too — anything trading citation accuracy for convenience got rejected.

DECISION 02

Nothing is written until you confirm

The natural way to build ingestion is to save the row, then enrich it in the background. I built the opposite: `/analyze` performs every AI operation against an in-memory buffer and persists nothing at all; `/commit` is the only endpoint that writes.

Two reasons. An abandoned add should leave nothing behind — half the point of this product is not accumulating junk. And the duplicate check is only useful *before* you save; telling someone they've just created a duplicate is a notification, not a feature.

DECISION 03

No LinkedIn scraping, even though it's the biggest source

Most of what I save is on LinkedIn, and automating that import would be the single most useful feature. Scraping your own saved posts violates LinkedIn's terms of service, so the app takes a pasted URL or pasted text and nothing else.

This costs real convenience. I wrote it into the brief as a locked constraint before building so it couldn't be quietly relitigated at 1am when the manual flow felt tedious.

DECISION 04

Cutting a feature I'd already specced

Resurfacing was core feature #7 in my own brief: items untouched for 14 days appear in a "revisit" widget. Six of the seven features shipped. I cut the seventh.

Not because it was hard — it was the cheapest thing left, two rules over a timestamp. I cut it because **surfacing items on an age rule guesses at intent the product has no evidence for.** "You saved this a fortnight ago" isn't a reason to read something. Retrieval already answers "find the right thing fast"; a widget nagging about old saves answers a question nobody asked. It goes back on the roadmap when usage shows people hunting for things they forgot they had — and I've written down what that evidence would look like.

DECISION 05

A hard $0 budget, treated as a design constraint

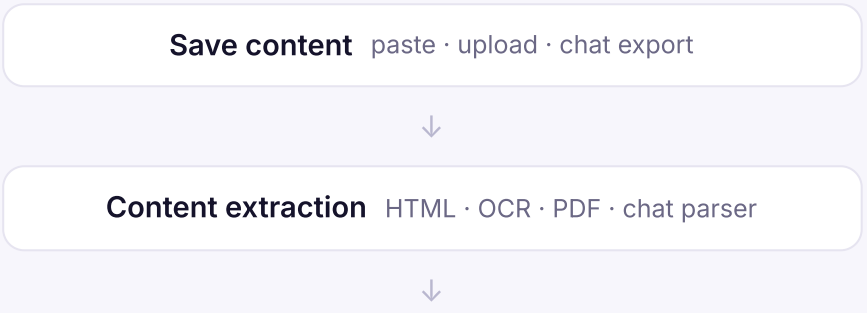
Every piece runs on a free tier: Supabase for Postgres, pgvector, auth and storage; Vercel for hosting and serverless functions; Google Gemini for generation and embeddings; OCR and PDF rendering in the browser, where they cost nothing.

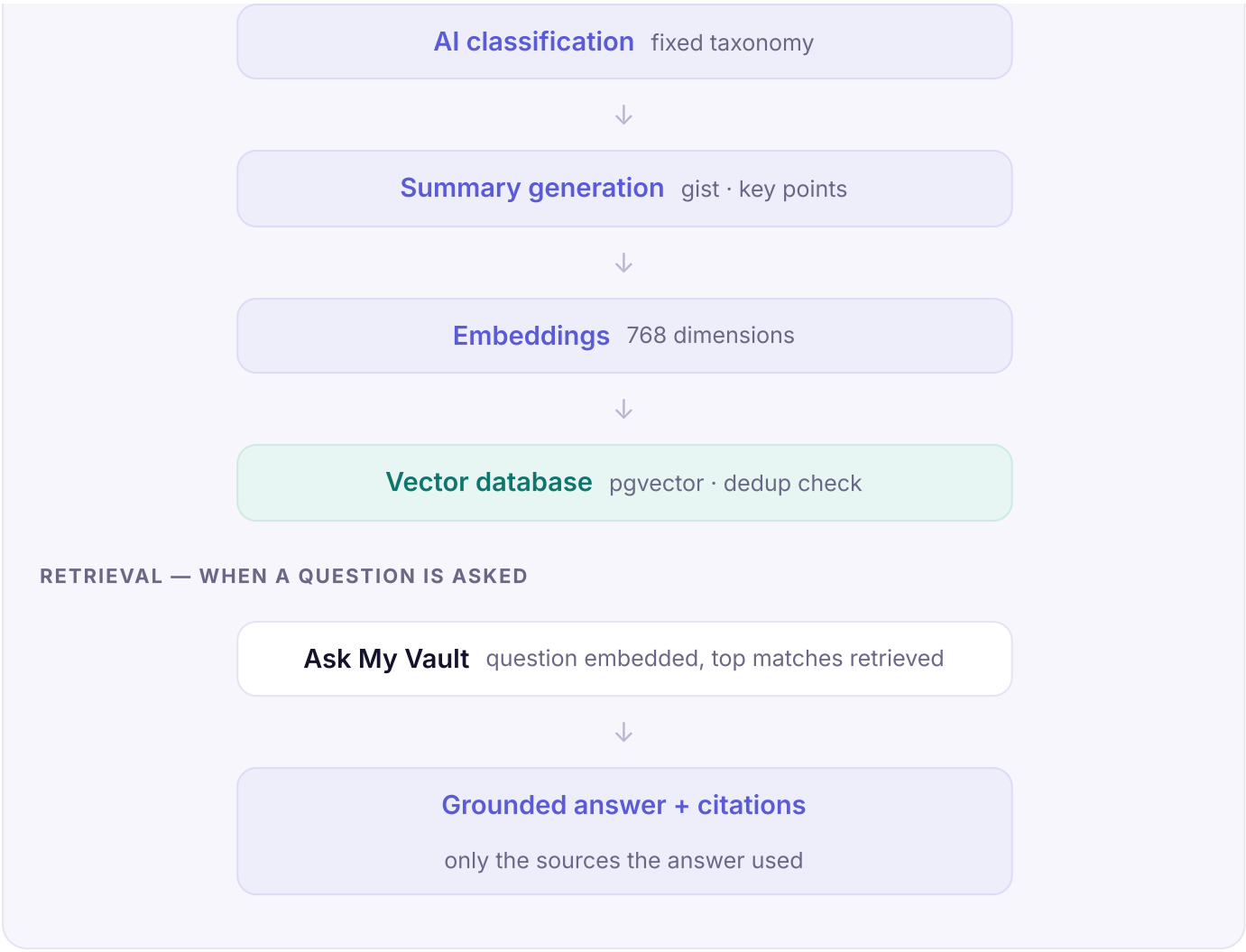
That constraint drove architecture rather than just billing. Uploads cap at 4MB because a serverless function rejects a larger body before my code runs, so a higher limit would only turn a clear message into an opaque failure. A daily cron pings the database because free Supabase projects pause when idle — and a portfolio link is most likely to be opened by exactly the person you least want waiting for a cold start.

AI architecture

Two AI paths, deliberately separated: one runs when content arrives, the other when a question is asked. They share an embedding model and nothing else.

INGESTION — WHEN CONTENT ARRIVES





What each stage does

- **Extraction is per type** — sanitised HTML for a note, Tesseract.js OCR in the browser for a screenshot, `pdf-parse` for a document plus pdf.js rasterising page 1 as cover art, a parser that splits a chat export into one item per message, and an HTML fetch with metadata parsing for a link.
- **Classification and summarisation are one call** — Gemini returns strict JSON: category from the fixed list, a short title and subtitle, a 1–2 line summary and 3–5 key points. The response is defended rather than trusted: malformed JSON is recovered, arrays coerced, and an empty response raises instead of saving a blank.
- **Duplicate detection reuses the embedding** — cosine similarity against existing items, surfaced *before* saving, when it can still change your mind.
- **Nothing is written until the review step is confirmed.** Everything above runs against a buffer; abandon at review and no row was ever created.
- **Only referenced sources become citations.** Retrieval returns the top matches; the answer cites some of them. Attaching all would inflate the "used N sources" claim into something untrue.
- **Citations keep their index and their passage**, so a reopened conversation still agrees with its own text — and the exact passage an answer leaned on stays checkable.

Designing for the model failing

The model is treated as an unreliable dependency rather than a given, because in practice it is.

FAILURE	RESPONSE
Model overloaded (503)	Retry with backoff, then fall back across models — another lightweight model first, full Flash last, because its free-tier allowance is roughly 20 requests/day
Daily quota exhausted (429)	A distinct error, because the advice differs: wait a minute versus wait until tomorrow
Malformed JSON	Recovered field by field rather than discarding the whole response
Classification fails entirely	The save still proceeds — the item lands with editable fields. The AI is an accelerator, not a gate
Embedding fails	Retried, because an item without a vector is silently invisible to search and skips dedup — worse than a visible missing summary

> **THE PRINCIPLE UNDERNEATH**
>
> Every one of these degrades toward *the user still gets their content saved*. The feature that must never degrade is the citation: an answer that can't be checked is worse than no answer, because it looks the same as one that can.

What broke, and what I changed

Four failures worth keeping, because each one changed how I think about shipping AI features.

– The app blamed the user for Google's outage

Adding content started failing with *"The AI couldn't read this one."* The content was fine. Google's `gemini-flash-lite` was returning `503 – high demand` on five consecutive calls.

Three separate mistakes compounded: the code used one model with no fallback, gave up on the first failure with no retry, and logged nothing — so a sustained upstream outage was indistinguishable from unreadable content, with no trace anywhere to tell them apart. Worst of all, the message blamed the user's input for someone else's traffic spike.

Now: retry with backoff, a fallback chain across models, every failure logged, and a distinct error for "the service is busy" versus "you're out of quota" — because the advice differs. The fallback order is deliberate: another lightweight model comes before full Flash, which allows only ~20

requests/day on the free tier, so leading with it would trade a short outage for an exhausted daily quota.

– A citation that pointed at the wrong number

An answer's inline markers count an item's position in the *retrieved* set, so a turn citing only the fourth match legitimately reads `[4]`. But reopening a saved conversation renumbered the sources from 1 — so the text said `[4]` beside a source card labelled `1`.

Worse, the restored view showed the item's *summary* in place of the passage the answer had actually used. For a product whose whole promise is checkable answers, a citation you can't verify is the failure that matters most.

Now: citations are stored with their index and the retrieved passage, not just an item id. The item itself is still looked up live, so edits show current and deletions drop out cleanly.

– A bug that only production would have found

Between analysing and saving, extracted text is held in memory. If that cache misses, commit re-extracts — sensible. But it kept only the text and discarded the metadata alongside it, which meant a note could save **with no body at all**, a link with no preview image, a PDF with no page count.

Locally this fired only if you left the review step open for fifteen minutes. On serverless, analyse and commit aren't guaranteed to hit the same instance — the rare path becomes the common one. I found it while reading the code for deployment, before it ever ran.

– Every uploaded file was publicly readable

The storage bucket was public. Any PDF or screenshot was readable by anyone holding the URL, no session required. Harmless while it ran on my laptop; not harmless the moment it had a public address.

Now: the bucket is private and every file link is signed with a two-hour expiry. The path parser reads both the old public URLs and the new object paths, so flipping it needed no data migration.

What I'd take to the next build

Six things I believe more strongly now than when I started, each one paid for by something in this project going wrong.

1. An AI feature's error message is part of the feature

When Gemini went down, the app said *"The AI couldn't read this one."* The content was perfectly readable; Google was overloaded. That sentence sends a user off editing input that was never the problem. Failure copy has to name the layer that actually failed — and if the system can't tell which layer that was, that's a logging gap, not a wording one.

2. A citation is a promise, and the bar is verification

Citations that looked right were subtly wrong twice: renumbered on reopen so the text disagreed with its own sources, and showing an item's summary in place of the passage the answer used. Neither would fail a test or look broken. For a product whose whole claim is grounded answers, **"looks cited" is not the standard — "can be checked" is.**

3. Constraints did more for scope than any prioritisation framework

A hard $0 budget and LinkedIn's terms of service each removed whole branches of the tree before I could argue with myself about them. Writing them down as locked decisions before building meant they held at 1am when the manual paste flow felt tedious. **The constraints I wrote down survived; the preferences I merely held did not.**

4. Cutting a specced feature is a decision, not an admission

Resurfacing was in my own brief and was the cheapest thing left to build. I cut it because surfacing items on an age rule guesses at intent I had no evidence for. The uncomfortable part wasn't the analysis — it was that I'd already written it down, and written-down plans acquire gravity. Being able to say *why* it's cut, with the evidence that would bring it back, is worth more than shipping it.

5. Working locally proves less than it feels like it proves

One bug would have corrupted saves in production while being nearly invisible in development: between analysing and saving, a cache miss dropped the metadata and could store a note with no body. Locally it needed a fifteen-minute pause to trigger. On serverless, where consecutive requests hit different instances, it becomes the normal path. **Deployment is a different environment, not just a different address.**

6. Security posture belongs before the URL, not after

Every uploaded file sat in a public bucket — fine on a laptop, not fine the moment the app had a public address. Making it private and signing every link took under an hour, and it was the thing I'd have been most embarrassed to be asked about. **The moment something becomes reachable is the moment its weakest assumption becomes a real one.**

THE ONE THAT GENERALISES

Most of these are the same lesson wearing different clothes: **an AI product's hard parts sit around the model, not in it.** The prompt was a day's work. Retries, fallbacks, honest failure states, verifiable citations, and knowing what to do when the model returns nonsense — that was the rest of the week, and it's what decides whether anyone trusts the output.

What I'd measure, and what I'd ask

STATED PLAINLY

This launched in August 2026 and has **one real user: me.** I have no usage data and no user feedback yet. Everything below is what I've set up to learn, not what I've learned. I'd rather show the plan than invent results.

The four numbers I care about

QUESTION	MEASURE	WHY THIS ONE
Does saved content get used?	% of saves opened at least once within 30 days	The core claim. If this doesn't beat the status quo of roughly zero, nothing else matters.
Does retrieval actually work?	% of vault queries where a cited source gets opened	An answer nobody clicks into is a plausible-sounding answer, which is the failure mode I'm most afraid of.
Is it faster than the old way?	Time from question to finding the right save	The product only earns its place if it beats scrolling LinkedIn saves.
Is the AI trusted or overridden?	% of AI-suggested fields edited at review	A high edit rate means the classifier is wrong. A zero edit rate probably means nobody's reading it.

The five questions I'd put to users

1. Show me the last five things you saved for prep. Where are they? *(Tests whether "scattered" is real or just my problem.)*
2. When did you last go looking for something you'd saved? Walk me through what you did.
3. What did you do when you couldn't find it? *(Re-searching Google instead is the strongest signal that saved effort is being wasted.)*
4. Here's an answer citing three of your saves. Do you trust it? What would make you check?

5. You haven't opened this in three weeks. Should the app have told you? *(The question that decides whether resurfacing comes back.)*

> **WHAT WOULD CHANGE MY MIND**
>
> If three of five describe hunting for something they knew they'd saved and giving up, **resurfacing goes back in** — and it should be triggered by what they were looking for, not by how old it is.
>
> If nobody clicks a citation, the trust model is wrong and the answer format needs rebuilding around the source rather than the prose.

How it's built

Frontend	React (Vite) + Tailwind, deployed on Vercel
API	Express, running as a Vercel serverless function on the same domain — so the frontend and API are same-origin and no CORS exists in production
Database	Supabase Postgres with `pgvector` for similarity search
Auth	Supabase Auth — Google plus email/password; every table carries `user_id` and row-level security
AI	Gemini for classification, summarisation and answers; `gemini-embedding-001` at 768 dimensions for retrieval
In the browser	Tesseract.js for screenshot OCR, pdf.js to rasterise a PDF's first page — both free and off the server

Two choices worth calling out. **The API is a serverless function on the same domain as the frontend**, so production has no cross-origin requests at all — CORS exists only for local development. And **OCR and PDF rendering run in the browser**, which keeps them off the server's clock and off the bill entirely.

Product Requirements – MVP

This is the brief the build was actually run against, written before the first commit rather than reconstructed afterwards. It's the reason the scope decisions above have dates and rationale attached — they were argued once, written down, and then honoured.

1. Problem & opportunity

Self-directed learning material is saved across LinkedIn, group chats and bookmarks with good intent, then rarely revisited. Even on a revisit it's hard to judge relevance without re-reading, or to find the right item at all. Saved effort quietly gets wasted. The MVP addresses this for Product Management learners, whose content types and vocabulary are predictable enough to classify well — the initial learning domain rather than the intended limit.

This has an architectural consequence: summaries are stored *alongside* full content rather than generated on demand, and "find the right thing fast" is prioritised over "store everything" — a plain folder app already does the latter.

2. Target user

Long-term: anyone who saves content to learn from later — self-directed learners, career switchers, people ramping into a new role or domain.

Initial audience (MVP): Product Management learners. A large, well-defined first user segment with predictable content types and a shared vocabulary, which is what makes a fixed taxonomy and an accurate classifier possible at all.

Deliberately not yet: other roles and domains. Serving them is a taxonomy and onboarding problem rather than an architectural one — see the roadmap.

3. Goals and non-goals

GOALS	NON-GOALS
One place that accepts every format prep content arrives in	Being a general-purpose read-later app
Judge an item's relevance without re-reading it	Replacing the source platforms
Answer questions across the whole library, with citations	Answering from general knowledge rather than the user's saves
Catch near-duplicates before they accumulate	Collaboration, sharing, multi-user features

4. Scope decisions (locked before building)

- **No live LinkedIn scraping.** Manual paste of post text or URL only — scraping saved posts violates LinkedIn's ToS.
- **No live WhatsApp listening.** The user exports a chat with WhatsApp's own feature and uploads the `.txt`, which is parsed into individual items.
- **Single user for the MVP.** No sharing — but every table carries `user_id` from day one so multi-user isn't a rewrite.
- **Resurfacing cut entirely** (2026-08-06, after the other six features shipped). See Decision 04.

5. Feature scope

#	FEATURE	STATUS
1	Manual ingestion — text, link, image (OCR), PDF, WhatsApp export, interview question, job posting	Shipped
2	Categorisation & summarisation on ingest	Shipped
3	Embedding generation into pgvector	Shipped
4	Library — list, filter, search, sort	Shipped
5	RAG chatbot with citations back to the source item	Shipped
6	Duplicate detection above a similarity threshold	Shipped
7	Resurfacing widget	**Cut** — see Decision 04

6. Data model

TABLE	PURPOSE
`items`	Source type, raw and extracted content, summary, key points, category, timestamps, and type-specific columns (page count, word count, job fields)
`embeddings`	Chunk text plus its 768-dimension vector, per item
`item_categories`, `tags`	Multi-category filing; tags also carry favourites
`duplicates`	Near-duplicate pairs with the cosine score that flagged them

TABLE	PURPOSE
`annotations`	Highlights and notes, anchored by character offset with quote and surrounding context so they survive edits
`chat_sessions`, `chat_queries`	Conversations, structured answers, and citations with their index and retrieved passage

7. Taxonomy

Interview Questions (Product Sense · RCA · Metrics · Strategy · Behavioural) · Job Postings · Application Tips · Frameworks · Industry News · Other.

8. Explicitly deferred

- LinkedIn or WhatsApp live sync
- Multi-user accounts and sharing
- Resurfacing of any kind, rule-based or predictive

9. What this project demonstrates

- **Problem definition** — diagnosing why saved content goes unused, and rejecting the obvious "better folders" answer
- **End-to-end AI product design** — ingestion, review, retrieval and verification as one experience
- **Designing for trust** — citations that resolve to the exact passage used, and answers that degrade honestly when the model fails
- **Independent delivery** — idea to deployed product, including auth, security posture and a live demo
- **MVP scoping and tradeoffs** — a deliberate first user segment, four locked constraints, one specced feature cut with its reasoning
- **RAG retrieval workflow** — extraction, embeddings, vector search and grounded generation, built rather than described
- **Feasibility against user value** — a $0 architecture, browser-side OCR, and limits chosen to fail with a clear message

10. Roadmap

Not built, and each one listed with the reason it isn't in the MVP rather than as a wish.

Next — the friction the MVP still leaves in

WHAT	WHY IT MATTERS
Share directly from any app	Every save is currently copy, switch app, paste. The mobile share sheet is where saving actually happens, and removing that step matters most of anything here — the premise is that saving stays effortless.
First-run onboarding	A new account opens onto an empty vault — nothing to search, nothing to ask. Onboarding should land one real item and one answered question before asking for trust.
Export the vault	What's saved here can only be read here. Export makes the tool safe to adopt, and belongs *inside* the reset and delete flows — a download offered before destroying a library is what makes the action reversible.
Browser extension	The desktop half of the same problem the share sheet solves.

Then — beyond the first domain

WHAT	WHY IT MATTERS
Any role, any domain	Nothing in ingestion, embedding, retrieval or citation is domain-specific — only the taxonomy is. Making categories configurable, or inferred from what someone saves, turns the MVP into a general learning system. Needs onboarding that asks what you're learning, and a classifier that works without a hand-written category list.
Practice mode	The saves already hold questions, frameworks and worked answers. Turning them into drills and spaced repetition is what separates a library from a study tool — and the most natural reason to return.
Collections	Grouping saves into a plan ("week 3: metrics") gives the vault a shape that a flat list and a category filter can't.

Later — needs other people in it

WHAT	WHY IT MATTERS
Shared vaults	Study groups already share this material in group chats. A shared vault is where the export feature and multi-user meet.

WHAT	WHY IT MATTERS
Anonymised "what's trending"	The network effect available here: the same embeddings that power personal search would cluster across accounts to show what a group is studying. Needs multi-user, a privacy model, and enough users for an aggregate to mean anything.

SEQUENCING

Next reduces friction on a product that already works; **Then** widens who it works for; **Later** needs users to connect, so it can't be pulled forward however appealing. The one thing I would not build without evidence is resurfacing — see Decision 04.

PM Content Vault — built and shipped as a solo project, 2026.
Live app (https://pm-content-vault.vercel.app) · Source (https://github.com/RADHIKA2909/pm-content-vault) · Demo login `demo@pmvault.app` / `PMVaultDemo2026!`